Retail Demand Forecasting

Walmart Store Sales — End-to-End Machine Learning Process Flow Document

July 2026

Contents

1. Project Overview

1.1 Business Problem

Walmart operates hundreds of stores across multiple departments, and weekly sales volumes fluctuate due to holidays, promotional markdowns, store characteristics, weather, and macroeconomic indicators. Reliable demand forecasts at the **Store–Department–Week** grain are required to optimize inventory, staffing, and promotional planning while minimizing stock-outs and overstock costs.

1.2 Objective

Build, validate, and compare multiple regression models capable of forecasting `Weekly_Sales` for every Store–Department combination, using historical sales, store metadata, and exogenous economic/weather signals. The final deliverable is a production-ready, serialized model pipeline together with a documented, reproducible process flow.

1.3 Dataset Summary

Source File	Description
`train.csv`	Historical weekly sales per Store, Department, Date, IsHoliday
`test.csv`	Future weeks requiring prediction (same schema, no `Weekly_Sales`)
`features.csv`	Store-level exogenous signals: Temperature, Fuel_Price, CPI, Unemployment, MarkDown1–5, IsHoliday
`stores.csv`	Static store metadata: Store Type (A/B/C) and Size

After merging, the working dataset contains **421,570 records across 16 raw features**, spanning **143 unique weekly dates**, suitable for chronological, time-series-aware modeling.

1.4 Technology Stack

Layer	Tools / Libraries
Data manipulation	pandas, NumPy
Visualization	matplotlib, seaborn
Preprocessing	scikit-learn (`Pipeline`, `ColumnTransformer`, `OneHotEncoder`)
Modeling	LinearRegression, RandomForestRegressor, XGBRegressor
Hyperparameter tuning	Optuna
Explainability	Random Forest native feature importance
Serialization	Joblib

1.5 Exploratory Data Analysis Highlights

Exploratory analysis on the cleaned dataset informed the feature engineering and modeling decisions described in later sections. Selected findings:

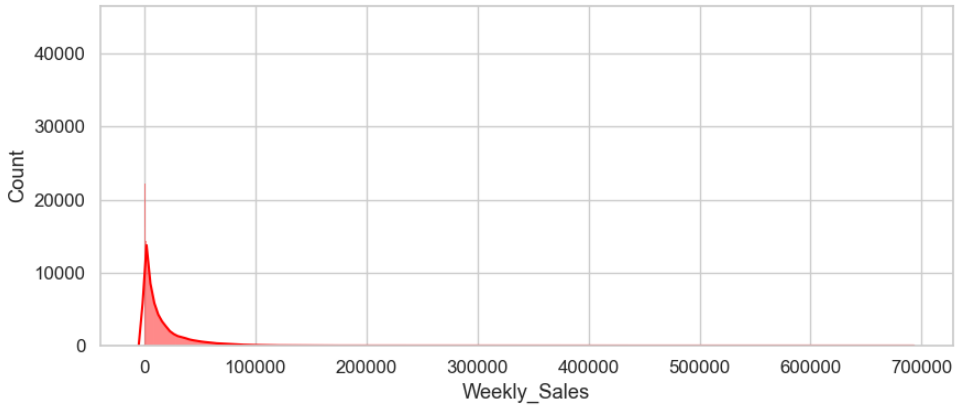


Figure 1: Distribution of Weekly Sales — right-skewed with high-value outliers

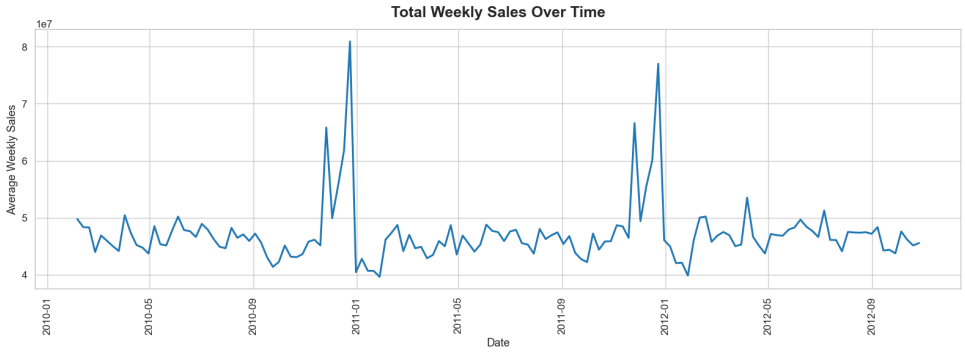


Figure 2: Aggregate Weekly Sales Over Time — seasonal peaks near late-year holidays

Figure 3: Average Weekly Sales by Store — store-level heterogeneity supports Store/Size features

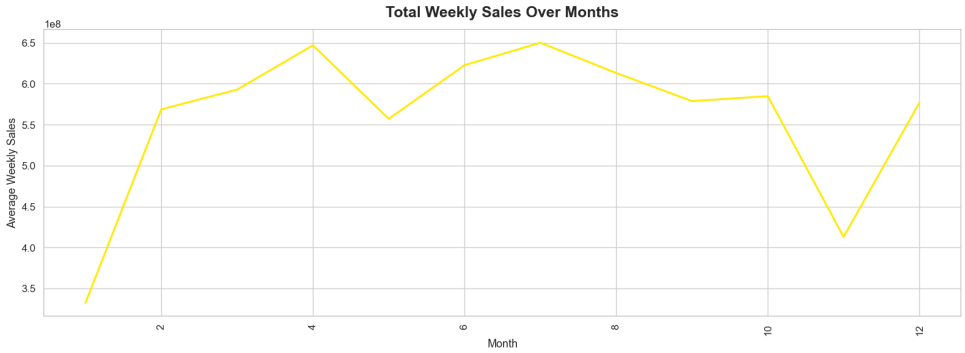


Figure 4: Monthly Sales Trend — recurring seasonal uplift

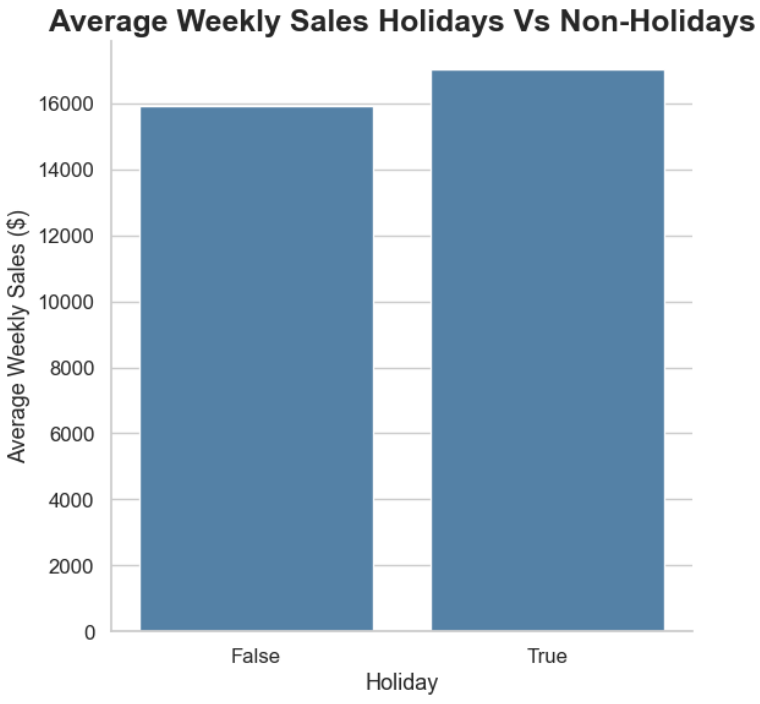


Figure 5: Average Sales: Holiday vs. Non-Holiday Weeks

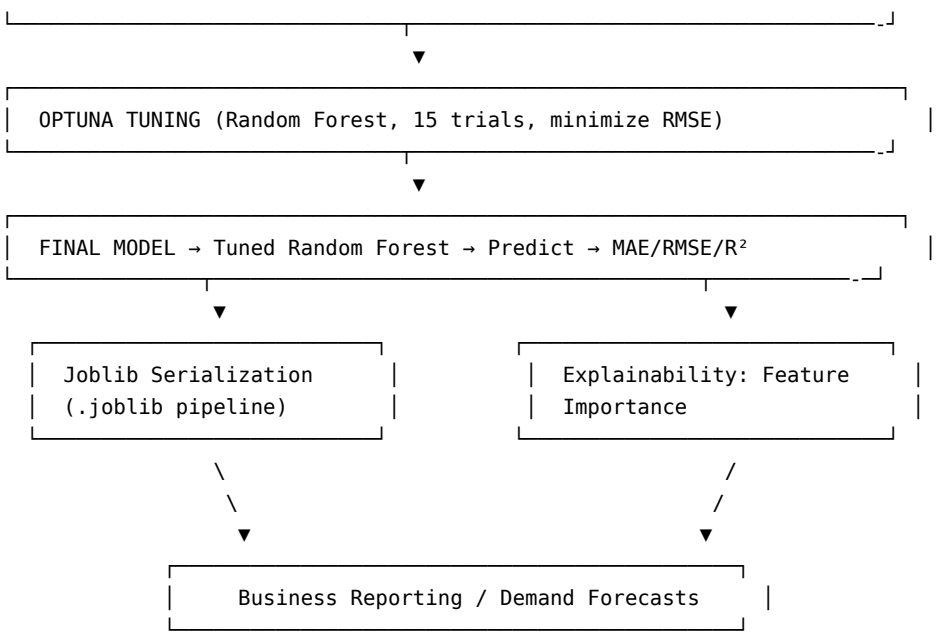
2. Process Flow Diagram

2.1 Mermaid Flowchart — End-to-End Architecture

```
flowchart TB
    A[("train.csv / features.csv\nstores.csv / test.csv")] --> B["Data Ingestion\nSchema + Shape Validation"]
    B --> C["Merge on Store, Date, IsHoliday"]
    C --> D["Data Cleaning\nDtype fix, MarkDown NaN to 0\nDuplicate & Range Checks"]
    D --> E["Feature Engineering\nYear, Month, WeekOfYear, Quarter\nLag_1, Rolling_Mean_4\nTotal_MarkDown, Holiday Fl
    E --> F["Chronological Split\nTrain vs Last 26 Weeks Validation"]
    F --> G["Pipeline + ColumnTransformer\nOneHotEncoder on Type"]
    G --> H["Model Training\nLinear Regression / Random Forest / XGBoost"]
    H --> I["Optuna Hyperparameter Tuning\n(Random Forest, 15 trials)"]
    I --> J["Tuned Random Forest\nFinal Model"]
    J --> K["Validation Prediction\nMAE / RMSE / R2"]
    K --> L["Model Serialization\nJoblib (.joblib)"]
    K --> M["Explainability\nFeature Importance"]
    L --> N["Business Reporting\nDemand Forecasts"]
    M --> N
```

2.2 ASCII Process Flow Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│  RAW DATA: train.csv | test.csv | features.csv | stores.csv          │
└──────────────────────────────┬───────────────────────────────────────┘
                               │  schema + shape validation
                               ▼
┌──────────────────────────────────────────────────────────────────────┐
│  MERGE  →  Store + Date (features)  →  Store (stores)                │
│  Resolve duplicate IsHoliday_x / IsHoliday_y → merged_train.csv      │
└──────────────────────────────┬───────────────────────────────────────┘
                               ▼
┌──────────────────────────────────────────────────────────────────────┐
│  DATA CLEANING                                                       │
│  - Date → datetime      - MarkDown1-5 NaN → 0                        │
│  - Duplicate check      - Negative Weekly_Sales investigation        │
│  - Range validation (Fuel_Price, Size, Temperature)                  │
└──────────────────────────────┬───────────────────────────────────────┘
                               ▼
┌──────────────────────────────────────────────────────────────────────┐
│  FEATURE ENGINEERING                                                 │
│  Year | Month | WeekOfYear | Quarter | Lag_1 | Rolling_Mean_4        │
│  Total_MarkDown | Holiday Flag (IsHoliday)                           │
└──────────────────────────────┬───────────────────────────────────────┘
                               ▼
┌──────────────────────────────────────────────────────────────────────┐
│  CHRONOLOGICAL SPLIT → Train (all but last 26 weeks) | Validation (26w) │
└──────────────────────────────┬───────────────────────────────────────┘
                               ▼
┌──────────────────────────────────────────────────────────────────────┐
│  PIPELINE  →  ColumnTransformer(OneHotEncoder[Type] + passthrough)   │
└──────────────────────────────┬───────────────────────────────────────┘
                               ▼
┌──────────────────────────────────────────────────────────────────────┐
│  MODEL TRAINING → LinearRegression | RandomForest | XGBoost          │
```


OPTUNA TUNING (Random Forest, 15 trials, minimize RMSE)
FINAL MODEL → Tuned Random Forest → Predict → MAE/RMSE/R²
Joblib Serialization
(.joblib pipeline)
Explainability: Feature
Importance
Business Reporting / Demand Forecasts

3. Data Ingestion

3.1 Loading Raw Datasets

Four CSV files are loaded independently using pandas: `train.csv`, `test.csv`, `features.csv`, and `stores.csv`. Each file is read via `pd.read_csv()` from a local `data/raw/` directory, keeping raw source data immutable and separate from any derived artifacts.

3.2 Initial Validation

Before merging, each dataset is inspected for structural consistency:

- **Row counts** of `train_df`, `test_df`, and `features_df` are compared to confirm that `features.csv` covers the full date range represented in both train and test partitions.
- **`.head()`** and **`.info()`** are used to visually and structurally confirm column names, data types, and non-null counts across all four sources.

3.3 Schema Validation

Each file carries a distinct but overlapping schema:

File	Key Columns	Grain
`train.csv` / `test.csv`	Store, Dept, Date, IsHoliday, (Weekly_Sales)	Store–Dept–Week
`features.csv`	Store, Date, Temperature, Fuel_Price, MarkDown1–5, CPI, Unemployment, IsHoliday	Store–Week
`stores.csv`	Store, Type, Size	Store

The shared keys `Store` and `Date` (plus `Store` alone for `stores.csv`) define the join logic used in the merge step.

3.4 Merging Datasets

Merging is performed as a two-step **left join**, preserving every row of the sales tables:

```
train_df = train_df.merge(features_df, on=["Store", "Date"], how="left")
train_df = train_df.merge(stores_df, on="Store", how="left")
```

The same sequence is applied to `test_df` to guarantee schema parity between training and inference data. Because both `train.csv` and `features.csv` contain an `IsHoliday` column, the merge produces two columns — `IsHoliday_x` (from sales data) and `IsHoliday_y` (from features data). Since both represent the same calendar holiday flag, the redundant `IsHoliday_y` column is dropped, and `IsHoliday_x` is later renamed to `IsHoliday` during cleaning.

3.5 Final Merged Dataset

The result is a single, model-ready flat table containing sales, holiday, promotional, economic, weather, and store-attribute fields at the Store–Dept–Week grain. This merged dataset is persisted to `data/merged_data/` and serves as the single source of truth for all downstream cleaning and feature engineering.

Resulting shape: 421,570 rows × 16 columns (training partition).

4. Data Cleaning & Transformation

4.1 Data Assessment

A structured assessment precedes any modification:

- `.shape`, `.info()`, `.describe()` (numeric and categorical) establish baseline statistics.
- `.isnull().sum()` quantifies missing values per column.
- `.duplicated().sum()` checks for exact-row duplication.
- `.nunique()` profiles cardinality of categorical fields (e.g., `Type` has 3 unique values: A, B, C).

Key findings: the dataset contains 421,570 records and 16 features with a mix of numerical, categorical, boolean, and datetime-like fields; no duplicate rows exist; `MarkDown1–MarkDown5` contain substantial missingness; `Weekly_Sales` contains a small number of negative values; the data spans 143 unique weekly dates, confirming suitability for time-series treatment.

4.2 Data Type Conversion

The `Date` column is loaded as a generic object/string and is explicitly converted to a proper datetime type to enable chronological operations (sorting, splitting, and date-part extraction):

```
df["Date"] = pd.to_datetime(df["Date"])
```

The holiday indicator is also renamed for clarity after the merge (`IsHoliday_x` → `IsHoliday`) and later cast to integer (`0`/`1`) prior to modeling so it can be consumed directly by numeric estimators.

4.3 Handling Missing Values (MarkDown1–MarkDown5)

The five markdown columns represent promotional discount amounts recorded only when a specific type of promotion was active. A missing value therefore does **not** indicate a data quality defect — it indicates the **absence of that promotion** during that store-week. Missing values are consequently imputed with `0` rather than a statistical estimate (mean/median), preserving the true business meaning:

```
markdown_cols = ["MarkDown1", "MarkDown2", "MarkDown3", "MarkDown4", "MarkDown5"]
df[markdown_cols] = df[markdown_cols].fillna(0)
```

4.4 Duplicate Checking

`df.duplicated().sum()` is re-verified after cleaning transformations; the dataset was confirmed to contain **zero duplicate records**, so no row-level de-duplication was required.

4.5 Validation of Numerical and Categorical Columns

Post-cleaning, `.info()` and `.describe()` are re-run to confirm dtype correctness (numeric columns are numeric, `Date` is datetime64, `Type` remains categorical/object) and to sanity-check ranges (e.g., no unexpected string values leaking into numeric columns).

4.6 Investigation of Negative Weekly Sales

A targeted check quantifies rows where `Weekly_Sales < 0`:

```
(df["Weekly_Sales"] < 0).sum()
```

A small number of negative values were identified. These are interpreted as **legitimate business events** — product returns or accounting corrections — rather than data-entry errors, and

were therefore **retained** rather than dropped or clipped, preserving the true distribution of demand (including returns) for the model to learn from. Similarly, sanity bounds were checked on `Fuel_Price` (< 0), `Size` (≤ 0), and `Temperature`, confirming no impossible values existed in these fields.

4.7 Time-Based Data Preparation

Because this is a forecasting problem, the dataframe is explicitly **sorted chronologically** by `Store`, `Dept`, and `Date` before any lag/rolling features are engineered — this ordering is a prerequisite for correct lag and rolling-window computation within each Store–Dept series.

4.8 Categorical Encoding — OneHotEncoder inside ColumnTransformer

The only nominal categorical feature retained for modeling is `Type` (store category A/B/C). It is one-hot encoded using scikit-learn's `OneHotEncoder(handle_unknown="ignore")`, wrapped inside a `ColumnTransformer` alongside a `"passthrough"` branch for the already-numeric/binary columns:

```
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), ["Type"]),
        ("num", "passthrough", numeric_cols + ["IsHoliday"]),
    ]
)
```

`handle_unknown="ignore"` guards against unseen store types appearing at inference time without raising an exception.

4.9 Why Pipeline and ColumnTransformer Were Used

- **Reproducibility & leakage prevention:** wrapping preprocessing and the estimator in a single `sklearn.pipeline.Pipeline` guarantees that the exact same transformation logic fitted on training data is applied to validation and future inference data — no manual re-application, no risk of accidentally fitting a transformer on validation data.
- **Separation of concerns:** `ColumnTransformer` cleanly routes categorical and numeric columns to the correct transformer without manual column slicing scattered through the codebase.
- **Single deployable artifact:** the fitted `Pipeline` (preprocessing + model) is serialized as one object, so production inference is a single `.predict()` call on raw-shaped input.
- **Algorithm-specific column strategy:** because Linear Regression and the tree-based models require different treatment of the markdown features (see Section 5), maintaining two distinct `ColumnTransformer` configurations behind two distinct pipelines cleanly isolates each model's preprocessing without code duplication or conditional branching inside the modeling loop.

5. Feature Engineering Pipeline

Feature engineering is performed **after** cleaning and **before** the chronological train/validation split, but lag and rolling features are computed strictly within each Store–Dept group to avoid cross-series leakage.

5.1 Engineered Features

Feature	Description	Business Rationale
Year	Calendar year extracted from `Date`	Captures long-term macro trend and year-over-year growth/decline in demand.
Month	Calendar month (1–12) extracted from `Date`	Captures monthly seasonality (e.g., back-to-school, holiday build-up).
WeekOfYear	ISO week number (1–52/53) extracted from `Date`	Retail demand is highly week-granular (e.g., Thanksgiving/Black Friday week); this is a finer-grained seasonality signal than month alone.
Quarter	Calendar quarter (1–4) extracted from `Date`	Aligns with retail fiscal/reporting cycles and broader seasonal shopping periods.
Lag_1	Prior week's `Weekly_Sales` for the same Store–Dept	Retail sales are highly autocorrelated week-to-week; last week's sales are typically the single strongest predictor of this week's sales.
Rolling_Mean_4	4-week rolling average of `Weekly_Sales` per Store–Dept	Smooths short-term noise and captures the recent demand trend/momentum, improving stability versus a single lag value.
Total_MarkDown	Sum of `MarkDown1` through `MarkDown5`	Consolidates fragmented promotional signals into one interpretable "total promotional intensity" metric, used specifically by the linear model to avoid multicollinearity.
Holiday Flag (IsHoliday)	Binary indicator (0/1) for holiday weeks	Holiday weeks (Super Bowl, Labor Day, Thanksgiving, Christmas) show materially different demand patterns and are a first-class driver of weekly sales spikes.

5.2 Why These Features Improve Retail Demand Forecasting

- **Calendar features (Year, Month, WeekOfYear, Quarter)** allow the model to learn recurring seasonal patterns without requiring an explicit time-series model — tree ensembles in

particular can split effectively on these to capture non-linear seasonal effects.

- **Lag_1 and Rolling_Mean_4** inject autoregressive signal directly into a tabular regression framework, letting standard regressors approximate time-series behavior (recency and momentum) without needing a dedicated ARIMA/Prophet-style architecture.
- **Total_MarkDown / MarkDown1–5** quantify promotional intensity, a direct, controllable driver of short-term demand spikes that the business can act on.
- **Holiday flag** captures one of the most important non-linear discontinuities in retail demand, where average sales during holiday weeks are measurably higher than non-holiday weeks (see EDA).

5.3 Post-Engineering Cleanup

Because `Lag_1` and `Rolling_Mean_4` require historical context, the first observations of each Store–Dept series do not have enough prior history and become `NaN`. These initial rows are explicitly identified and dropped (`dropna(subset=["Lag_1", "Rolling_Mean_4"])`) rather than imputed, since fabricating a "previous week's sales" value would introduce artificial signal.

6. Model Training & Validation

6.1 Time-Series-Aware Train/Validation Split

The final 26 weeks (approximately six months) of the dataset are reserved as a **chronological validation set**, with all earlier weeks used for training:

```
unique_dates = sorted(df["Date"].unique())
validation_dates = unique_dates[-26:]
train_df = df[~df["Date"].isin(validation_dates)]
val_df   = df[df["Date"].isin(validation_dates)]
```

6.2 Why Random Train-Test Splitting Was Not Used

A random `train_test_split` would shuffle rows across time, allowing the model to train on weeks that occur **after** the weeks it is validated on. This causes **temporal leakage**: the model could implicitly learn from future information (e.g., via `Lag_1`/`Rolling_Mean_4` computed from nearby dates) that would never be available in a genuine forecasting scenario. A strict chronological split ensures the validation set exclusively contains **future, unseen time periods**, giving an honest estimate of how the model would perform when deployed to forecast upcoming weeks.

6.3 Pipeline Architecture

Each model is wrapped in its own two-stage `Pipeline`:

```
Pipeline(steps=[
    ("preprocessor", ColumnTransformer(...)),
    ("model", <Estimator>)
])
```

This guarantees identical preprocessing logic is applied consistently at training, validation, and inference time.

6.4 ColumnTransformer Workflow — Model-Specific Feature Sets

Because `Total_MarkDown` is an exact linear sum of `MarkDown1–MarkDown5`, including both creates redundant, perfectly collinear information. Two distinct preprocessing/feature strategies are used:

Model	Markdown Features Used	Reason
Linear Regression	`Total_MarkDown` only (parts dropped)	Linear models are sensitive to multicollinearity; a single aggregate metric keeps coefficient estimation stable.
Random Forest	`MarkDown1–MarkDown5` (total dropped)	Tree splits are immune to collinearity; granular columns let the model discover which specific promotion types matter.
XGBoost	`MarkDown1–MarkDown5` (total dropped)	Same rationale as Random Forest.

Both `Type` (categorical) and the numeric/binary columns are routed through a shared `ColumnTransformer` pattern: `OneHotEncoder(handle_unknown="ignore")` for `Type`, and `"passthrough"` for the numeric feature block (including `IsHoliday`).

6.5 Models Trained

Three baseline regressors are trained and evaluated on the chronological validation set:

1. **Linear Regression** — interpretable baseline; establishes a linear performance floor.
2. **Random Forest Regressor** — non-linear ensemble, robust to feature scaling and collinearity, `n_jobs=-1` for parallel training.
3. **XGBoost Regressor** — gradient-boosted ensemble, trained with GPU acceleration (`device="cuda"`) for faster iteration.

Baseline Validation Results

Model	MAE	RMSE	R^2
Linear Regression	1561.52	3257.15	0.9782
Random Forest	1466.72	3049.20	0.9809
XGBoost	1565.60	3232.39	0.9785

Random Forest produced the strongest baseline performance across all three metrics, and was selected as the candidate for hyperparameter tuning.

6.6 Hyperparameter Tuning Using Optuna (Random Forest)

An Optuna study (`direction="minimize"`, objective = validation RMSE) searches over 15 trials across the following search space:

Hyperparameter	Search Range
`n_estimators`	100–600 (step 100)
`max_depth`	10–40
`min_samples_split`	2–10
`min_samples_leaf`	1–5
`max_features`	`sqrt`, `log2`, `None`

Each trial re-fits a complete `Pipeline` (preprocessor + `RandomForestRegressor`) with the sampled hyperparameters and scores it against the held-out chronological validation set, giving Optuna a direct, leakage-free optimization signal.

6.7 Final Tuned Random Forest Model

The tuning results were used to configure a refined final model, with several parameters manually adjusted from Optuna's raw optimum toward a more regularized configuration to reduce overfitting risk (fewer trees, shallower depth, larger minimum split/leaf sizes) while retaining Optuna's recommended `max_features="log2"`:

```
RandomForestRegressor(
    n_estimators=300,
    max_depth=20,
    min_samples_split=5,
    min_samples_leaf=2,
    max_features="log2",
    random_state=42,
    n_jobs=-1,
)
```

6.8 Training Pipeline Workflow Diagram

```
flowchart TB
    subgraph Data["Chronologically Split Data"]
      X["X_train / y_train\nX_val / y_val (last 26 weeks)"]
    end
    X --> CT["ColumnTransformer\nOneHotEncoder(Type) + passthrough numeric"]
    CT --> P1["Pipeline: Linear Regression\n(Total_MarkDown, parts dropped)"]
    CT --> P2["Pipeline: Random Forest\n(MarkDown1-5, total dropped)"]
    CT --> P3["Pipeline: XGBoost\n(MarkDown1-5, total dropped)"]
    P1 --> EV["Evaluate on Validation Set\nMAE / RMSE / R2"]
    P2 --> EV
    P3 --> EV
    EV --> SEL{"Best Baseline?"}
    SEL -->|Random Forest wins| OPT["Optuna Search\n15 trials, minimize RMSE"]
    OPT --> TUNED["Tuned RandomForestRegressor\nn_estimators=300, max_depth=20\nmin_samples_split=5, min_samples_leaf
    TUNED --> FINAL["Final Evaluation\nMAE 1408.16 / RMSE 2890.97 / R2 0.9828"]
```

7. Prediction & Post-Processing

7.1 Prediction on Validation Dataset

The tuned pipeline predicts `Weekly_Sales` for the held-out 26-week validation partition using the same `MarkDown1`–`MarkDown5` feature set it was trained on (`Total_MarkDown` dropped to match the tree-model `ColumnTransformer`):

```
pred = rf_tuned_pipeline.predict(X_val_rf)
```

7.2 Evaluation Metrics

Metric	Formula Basis	Purpose
MAE	Mean Absolute Error	Average magnitude of prediction error, in original sales units, robust to outliers.
RMSE	Root Mean Squared Error	Penalizes larger errors more heavily; highlights model robustness to demand spikes.
R^2	Coefficient of Determination	Proportion of variance in `Weekly_Sales` explained by the model.

Final Tuned Model Performance

Metric	Value
MAE	1408.16
RMSE	2890.97
R^2	0.9828

7.3 Final Model Selection

Across all baseline and tuned candidates, the **tuned Random Forest** achieved the lowest MAE, lowest RMSE, and highest R^2, confirming its superiority in capturing the non-linear interactions between historical sales, store characteristics, promotions, and macroeconomic factors:

Model	MAE	RMSE	R^2
Linear Regression	1561.52	3257.15	0.9782
Random Forest (baseline)	1466.72	3049.20	0.9809
XGBoost	1565.60	3232.39	0.9785
Random Forest (Tuned)	**1408.16**	**2890.97**	**0.9828**

The tuned Random Forest was selected as the production model.

7.4 Model Serialization Using Joblib

The complete fitted pipeline (preprocessing + estimator) is serialized with Joblib for downstream deployment/reuse:

```
import joblib
joblib.dump(rf_tuned_pipeline, "random_forest_model.joblib")
```

Serializing the full `Pipeline` object (rather than the bare estimator) ensures inference code only needs to call `.predict()` on raw-shaped feature input, since the `ColumnTransformer` step is bundled with the model.

7.5 Feature Importance

Native Random Forest feature importances were extracted from the fitted pipeline's `ColumnTransformer` output feature names and ranked:

```
feature_names = preprocessor.get_feature_names_out()
importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": rf_model.feature_importances_,
}).sort_values("Importance", ascending=False)
```

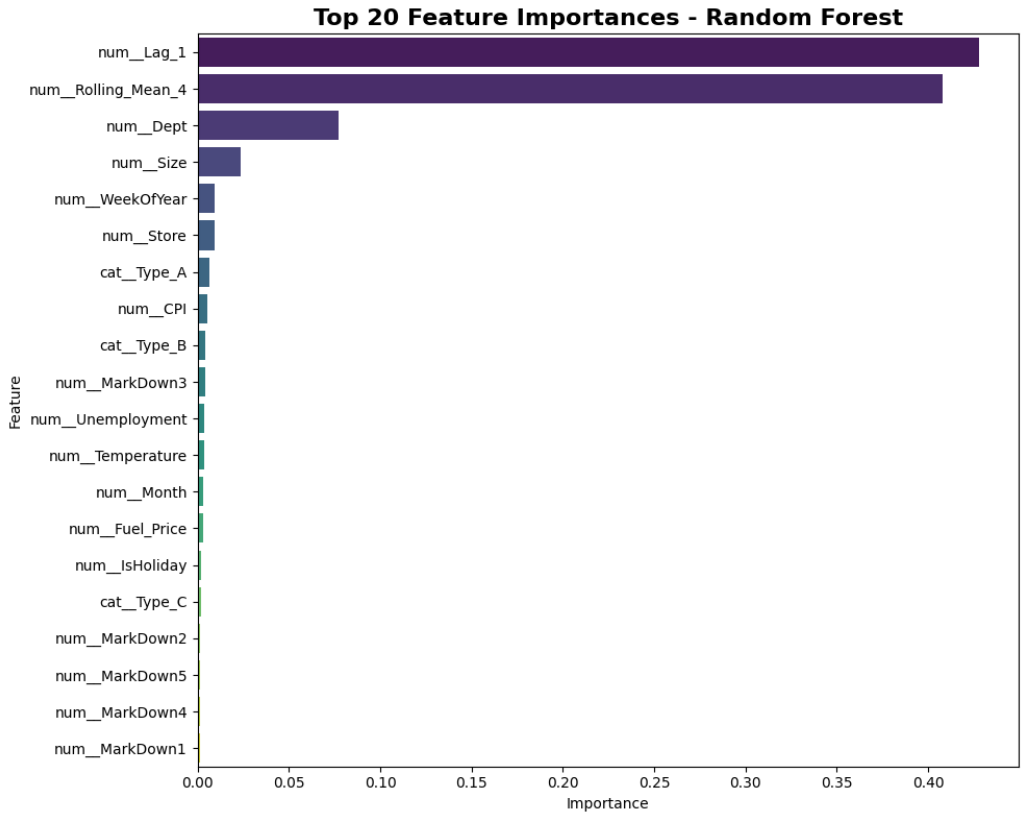


Figure 6: Top 20 Feature Importances — Random Forest

7.6 Business Interpretation of Predictions

- **`Lag_1`** is the single most influential feature — the prior week's sales is the strongest predictor of current-week sales, confirming strong short-term demand persistence.
- **`Rolling_Mean_4`** reinforces this by capturing recent trend/momentum, adding forecasting stability beyond a single lag point.
- **`Dept`** contributes strongly, since different departments (e.g., grocery vs. electronics) exhibit structurally different demand levels and seasonality.

- **`Size`** and **`Store`** capture store-level heterogeneity — larger stores and specific store identities carry distinct baseline demand levels.
- **Promotional (`MarkDown1–5`), economic (`CPI`, `Unemployment`, `Fuel_Price`), and weather (`Temperature`) variables** contribute comparatively smaller but non-trivial signal, providing contextual adjustment on top of the dominant autoregressive and categorical drivers.

These findings translate directly into business action: inventory planning should prioritize recent sales momentum (Lag_1/Rolling_Mean_4) per Store–Dept, with promotional and macroeconomic indicators used as secondary levers for fine-tuning forecasts around holidays and markdown events.

8. Conclusion

This project delivered a complete, reproducible, production-oriented retail demand forecasting pipeline for Walmart Store Sales data. Key outcomes:

- A clean, validated, merged dataset built from four raw sources with rigorous schema and quality checks.
- A feature engineering strategy that injects calendar seasonality and autoregressive signal (`Lag_1`, `Rolling_Mean_4`) into a standard tabular regression framework.
- A leakage-free, time-series-aware validation strategy (last 26 weeks held out) that realistically simulates production forecasting conditions.
- A systematic model comparison (Linear Regression, Random Forest, XGBoost) followed by Optuna-driven hyperparameter optimization, converging on a **tuned Random Forest** as the final model (MAE 1408.16, RMSE 2890.97, R^2 0.9828).
- A fully serialized, deployment-ready `Pipeline` artifact, paired with native Random Forest feature-importance analysis to support transparent, business-facing decision-making.

The resulting model and process flow are directly reusable: the same pipeline architecture can be retrained on updated weekly data, re-validated on the newest 26-week window, and redeployed with minimal code changes, making it suitable as a recurring, production-grade forecasting workflow.

End of Document

— Hemanth Korakoppula